



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Tema 3:

Lenguajes y Gramáticas Formales

Profesor :

Félix Márquez

Alumnos:

Shirley Cedeño V-30.413.183 S2

Nelson Bueno V-27.407.263 S2

Angel Rodriguez V-30.437.205 S1

Rayc Yanez V-28.215.618 S2

Ciudad Guayana, junio del 2026

Resumen Académico

El presente informe aborda el estudio teórico y práctico de los lenguajes y gramáticas formales, fundamentos esenciales en la arquitectura de compiladores. Inicialmente, se explora la Jerarquía de Chomsky y la relación intrínseca entre gramáticas y lenguajes, detallando el mecanismo lógico de derivación sintáctica. Asimismo, se desarrolla el modelado bidimensional a través de una Gramática Libre de Contexto aplicada a un sistema de dibujo mediante comandos atómicos, evidenciando el control sintáctico sobre trayectorias espaciales complejas. Posteriormente, se diagnostican y optimizan patologías estructurales comunes en las gramáticas formales, tales como la ambigüedad, la recursividad por la izquierda y el no determinismo (factorización), garantizando así la viabilidad del análisis sintáctico predictivo. Finalmente, el estudio desciende al nivel léxico para diseñar un motor de reconocimiento orientado a un subconjunto de la notación de ajedrez (PGN), aplicando la dualidad matemática entre las Expresiones Regulares y los Autómatas Finitos Determinísticos (AFD). Este enfoque integral demuestra cómo la abstracción matemática se traduce directamente en soluciones computacionales deterministas, eficientes y libres de ambigüedad.

Introducción

El desarrollo de software robusto y la construcción de compiladores requieren una comprensión profunda de cómo las máquinas procesan la información. A diferencia de los idiomas humanos, que están sujetos a la interpretación subjetiva y al contexto, los lenguajes de programación exigen un rigor matemático absoluto. Este nivel de precisión unívoca se alcanza mediante el estudio de los Lenguajes y Gramáticas Formales, disciplina que proporciona los cimientos teóricos necesarios para validar, estructurar y traducir secuencias de caracteres en instrucciones ejecutables.

El presente trabajo tiene como propósito fundamental aplicar estos conceptos teóricos a escenarios prácticos de la ingeniería informática. En la primera sección, se desglosa la Jerarquía de Chomsky y los mecanismos de derivación gramatical, estableciendo las leyes de generación estructural de los lenguajes. Seguidamente, se presenta el modelado de un lenguaje de dominio específico orientado al dibujo bidimensional, demostrando la capacidad de las Gramáticas Libres de Contexto para orquestar la generación de gráficos.

La tercera sección aborda la higiene gramatical, una fase de ingeniería crítica donde se identifican y corrigen malformaciones sintácticas que podrían colapsar un analizador o generar bucles infinitos en el procesamiento. Por último, la investigación aborda el análisis léxico puro, modelando un reconocedor de jugadas de ajedrez mediante Expresiones Regulares y Autómatas Finitos Determinísticos. En conjunto, este informe establece un puente directo entre la teoría matemática más abstracta y su implementación lógica en las fases iniciales de la compilación.

Fundamentos y Jerarquía (Teoría Aplicada)

Relación Gramática-Lenguaje: Concepto, Ejemplo y Mecanismo

El Concepto de la Relación: En las ciencias de la computación, un alfabeto (Σ) es un conjunto finito de símbolos, y una palabra o cadena es una secuencia finita de estos. Un lenguaje formal (L) es el conjunto matemático de todas las cadenas válidas bajo ciertas reglas.

La gramática formal (G), por su parte, es el motor generador. No es el lenguaje en sí, sino el mecanismo finito que define cómo construir un conjunto potencialmente infinito de cadenas válidas. Decimos que una gramática G define a un lenguaje L (denotado como $L(G)$) si y solo si el lenguaje está compuesto exactamente por todas las cadenas de terminales que la gramática puede generar. En el contexto de la compilación, esta gramática actúa como el esqueleto que define la sintaxis, permitiendo validar la estructura jerárquica del código.

Mecanismo de Generación: La Derivación

Formalmente, una gramática se define por una cuádrupla $G = (V_n, V_t, P, S)$, donde:

- **V_n :** Conjunto de variables o símbolos No Terminales (estructuras sintácticas intermedias).
- **V_t :** Conjunto de símbolos Terminales (los caracteres reales del alfabeto Σ).
- **P :** Conjunto finito de Reglas de Producción (de la forma $\alpha \rightarrow \beta$).
- **S :** El Símbolo Inicial ($S \in V_n$), desde donde comienza la evaluación.

El mecanismo mediante el cual se pasa de la gramática al lenguaje es la **derivación**. Consiste en aplicar recursivamente las reglas de producción: se toma una cadena, se busca un

símbolo no terminal y se reemplaza por el lado derecho de una producción aplicable. Este proceso se repite en pasos sucesivos ($\Rightarrow \dots \Rightarrow$) hasta que la cadena esté compuesta únicamente por símbolos terminales (V_t), momento en el cual la palabra oficialmente pertenece al lenguaje.

Ejemplo Práctico: Imaginemos un lenguaje simplificado para definir asignaciones de variables en programación, donde:

- Símbolo inicial: S
- Producciones:
 - $S \rightarrow =$
 - $\rightarrow x \mid y$
 - $\rightarrow 1 \mid 2$

Mecanismo en acción (Derivación de la cadena $x = 2$): $S \Rightarrow = \Rightarrow x = \Rightarrow x = 2$

Jerarquía de Chomsky

Propuesta por el lingüista Noam Chomsky en la década de 1950, esta jerarquía clasifica las gramáticas en 4 niveles (del Tipo 0 al Tipo 3). A medida que el número del tipo aumenta, las restricciones sobre las reglas de producción son más estrictas, pero el poder computacional necesario para reconocerlas disminuye, estableciendo una relación directa entre las reglas y la capacidad de la máquina.

A continuación se detallan los 4 tipos acompañados de su representación en BNF (Backus-Naur Form).

Nota técnica: El estándar BNF está diseñado intrínsecamente para expresar sintaxis libre de contexto (Tipo 2). Por lo tanto, para los Tipos 0 y 1 se ha

adaptado la notación conceptualmente con el fin de ilustrar las restricciones semánticas o de entorno que estas gramáticas exigen.

- **Tipo 0: Gramáticas No Restringidas (Lenguajes Recursivamente Enumerables)**

Estas no tienen restricciones morfológicas en sus reglas de producción. Cualquier cadena de no terminales y terminales puede transformarse en cualquier otra combinación. Son equivalentes en poder a una Máquina de Turing.

Ejemplo en Notación BNF (Adaptada conceptualmente): Se permite que el lado izquierdo tenga múltiples símbolos y cambie el contexto de forma libre.

```
;; Modifica el contexto de manera desestructurada
<A> <B> ::= <C> "x"
<C>      ::= "a"
```

- **Tipo 1: Gramáticas Sensibles al Contexto (Lenguajes Sensibles al Contexto)** Las producciones son de la forma $\alpha A \beta \rightarrow \alpha \gamma \beta$. El símbolo no terminal A solo puede ser reemplazado por γ si está rodeado por el "contexto" de α y β . Además, el lado derecho no puede ser más corto que el izquierdo ($|\alpha| \leq |\beta|$). Son procesados por Autómatas Linealmente Acotados.

Ejemplo en Notación BNF (Representando el contexto): Un caso clásico es el lenguaje matemático $L = \{a^n b^n c^n \mid n \geq 1\}$.

```
;; El reemplazo de un elemento depende de sus vecinos directos
<Contexto_A> <Variable> <Contexto_B> ::= <Contexto_A> "bloque_valido" <Contexto_B>
```

- **Tipo 2: Gramáticas Libres de Contexto (Lenguajes Libres de Contexto)** El lado izquierdo de cualquier regla de producción debe ser obligatoriamente un solo símbolo no terminal. No importa qué caracteres rodeen a este símbolo; la sustitución siempre es válida e independiente. Es el esqueleto que define la sintaxis de los lenguajes de programación modernos (estructuras anidadas, paréntesis, bloques de código) y se procesa mediante Autómatas de Pila (Parsers).

Ejemplo en Notación BNF (Estructura de un bloque condicional if-else):

```
<sentencia_if> ::= "if" "(" <condicion> ")" <bloque> <opcional_else>
<opcional_else> ::= "else" <bloque> | ""
<condicion>      ::= "true" | "false"
<bloque>         ::= "{" "instrucciones" "}"
```

- **Tipo 3: Gramáticas Regulares (Lenguajes Regulares)** Son las más restrictivas. Las reglas solo permiten un no terminal a la izquierda, y a la derecha un terminal seguido (o precedido) por máximo un no terminal (lineales a la derecha o a la izquierda). Son las que dan vida a las Expresiones Regulares (Regex) y se procesan eficientemente con Autómatas Finitos para el análisis léxico (identificación de tokens).

Ejemplo en Notación BNF (Definición léxica de un Identificador/Variable):

```
<identificador> ::= <letra> | <letra> <resto_id>
<resto_id>      ::= <letra> <resto_id> | <digito> <resto_id> | <letra> | <digito>
<letra>         ::= "a" | "b" | "c"
<digito>        ::= "0" | "1" | "2"
```

Derivación y Modelado (Caso Práctico: Dibujo con Genoma)

Gramática para dibujar

El presente apartado detalla el diseño de una Gramática Libre de Contexto (GLC) que modela herramientas de dibujo bidimensionales. Para evitar ambigüedades en la interpretación, se ha estructurado una traza de ejecución paso a paso (derivación por la izquierda) donde se especifica la regla de producción aplicada y la semántica geométrica exacta de la acción del "trazador" (análogo a un gráfico de tortuga).

Definición Formal de la Gramática

Toda gramática G se define matemáticamente como la cuádrupla $G = (V, \Sigma, P, S)$. A continuación, se definen los parámetros adaptados a nuestro lenguaje de modelado:

- Alfabeto de Símbolos Terminales (Σ): Los caracteres del genoma asignados a movimientos elementales. $\Sigma = \{a, c, g, t\}$.
 - a (Avanzar): Traza una línea recta de longitud K en la dirección actual.
 - c (Cruzar/Derecha): Rota el ángulo de dirección actual $+90^\circ$ (giro a la derecha).
 - g (Girar/Izquierda): Rota el ángulo de dirección actual -90° (giro a la izquierda).

- t (Trasladar/Retornar): Retrocede una longitud K sobre la línea trazada previamente sin alterar el ángulo de dirección. Es una función de memoria de posición para bifurcaciones.
- Conjunto de No Terminales (V): Las macro-instrucciones geométricas. $V = \{S, \text{Cuadrado}, \text{Arbol}, \text{Cubo}, \text{Cruz}, \text{Escalera}, L, D, I, R\}$.
- Símbolo Inicial (S): Punto de entrada del analizador.

Conjunto de Producciones (P)

Producción	Regla	Descripción Lógica
P1	$S \rightarrow \text{Cuadrado} \mid \text{Arbol} \mid \text{Cubo} \mid \text{Cruz} \mid \text{Escalera}$	Derivación principal de las 5 figuras exigidas.
P2	$\text{Cuadrado} \rightarrow L D L D L D L$	Ciclo cerrado: Lado, giro 90° , repetido 4 veces.
P3	$\text{Arbol} \rightarrow L I L R D D L$	Bifurcación: Tallo, rama izquierda, retroceso al centro, compensación de ángulo, rama derecha.
P4	$\text{Cubo} \rightarrow \text{Cuadrado} I L D L D L$	Proyección 2D: Cara frontal, sesgo de profundidad y aristas laterales/superiores.
P5	$\text{Cruz} \rightarrow L R D L R D L R D L$	Centro radial: Línea, retorno al origen, giro 90° , repetido 4 veces en bucle.
P6	$\text{Escalera} \rightarrow L D L I L D L I L$	Patrón zig-zag: Avance intercalado con giros alternos derecha/izquierda.
P7 a P10	$L \rightarrow a \mid D \rightarrow c \mid I \rightarrow g \mid R \rightarrow t$	Resolución léxica de los símbolos No Terminales de acción a los Terminales.

Cuadros de Derivación Explicativa (Leftmost Derivation)

Se aplica el reemplazo estricto del símbolo No Terminal ubicado más a la izquierda en cada paso de computación. El símbolo $\Rightarrow$ indica una derivación lógica hacia el siguiente estado.

Caso A: Cuadrado

Paso	Cadena Derivada	Producción	Acción Geométrica (Semántica)
1	$S \Rightarrow \text{Cuadrado}$	P1	Llamada a la función de la figura.
2	$S \Rightarrow L D L D L D L$	P2	Expansión de la plantilla sintáctica del polígono.
3	$S \Rightarrow a D L D L D L$	P7	Avanza: Dibuja el primer lado inferior (base).
4	$S \Rightarrow a c L D L D L$	P8	Gira Der: Se orienta hacia arriba.
5	$S \Rightarrow a c a D L D L$	P7	Avanza: Dibuja el lado vertical derecho.
6	$S \Rightarrow a c a c L D L$	P8	Gira Der: Se orienta hacia la izquierda.
7	$S \Rightarrow a c a c a D L$	P7	Avanza: Dibuja el lado superior (techo).
8	$S \Rightarrow a c a c a c L$	P8	Gira Der: Se orienta hacia abajo.
9	$S \Rightarrow a c a c a c a$	P7	Avanza: Dibuja el lado vertical izquierdo, cerrando el polígono.

Caso B: Árbol (Bifurcación)

Paso	Cadena Derivada	Producción	Acción Geométrica (Semántica)
1	$S \Rightarrow \text{Arbol}$	P1	Llamada a la función de la figura.
2	$S \Rightarrow L I L R D D L$	P3	Expansión de la plantilla sintáctica.
3	$S \Rightarrow a I L R D D L$	P7	Avanza: Dibuja el tronco principal.
4	$S \Rightarrow a g L R D D L$	P9	Gira Izq: Inclina el vector hacia la diagonal izquierda.
5	$S \Rightarrow a g a R D D L$	P7	Avanza: Dibuja la rama izquierda.
6	$S \Rightarrow a g a t D D L$	P10	Retorna: El puntero retrocede la distancia de la rama, ubicándose de nuevo en la punta del tronco, sin cambiar su ángulo (sigue mirando a la izq).
7	$S \Rightarrow a g a t c D L$	P8	Gira Der: Cancela la inclinación izquierda, mirando al frente.
8	$S \Rightarrow a g a t c c L$	P8	Gira Der: Inclina el vector hacia la diagonal derecha.
9	$S \Rightarrow a g a t c c a$	P7	Avanza: Dibuja la rama derecha, completando el árbol.

Caso C: Cubo (Perspectiva 2D)

Paso	Cadena Derivada	Producción	Acción Geométrica (Semántica)
1	$S \Rightarrow \text{Cubo}$	P1	Llamada a la función de la figura.
2	$S \Rightarrow \text{Cuadrado } I L D L D L$	P4	Expansión. Llama a un bloque complejo y luego proyecta la fuga visual.
3	$S \Rightarrow a c a c a c a I L D L D L$	P2, P7-8	Resolución Macro: Se ejecuta y dibuja la cara

			frontal completa (Caso A). El puntero queda en el vértice inicial mirando hacia abajo.
4	$S \Rightarrow a c a c a c a g L D L D L$	P9	Gira Izq: El puntero se orienta en diagonal (eje Z de fuga isométrico).
5	$S \Rightarrow a c a c a c a g a D L D L$	P7	Avanza: Traza la arista inferior de profundidad.
6	$S \Rightarrow a c a c a c a g a c L D L$	P8	Gira Der: El puntero se orienta hacia arriba.
7	$S \Rightarrow a c a c a c a g a c a D L$	P7	Avanza: Traza la arista vertical trasera (fondo de la cara lateral).
8	$S \Rightarrow a c a c a c a g a c a c L$	P8	Gira Der: El puntero se orienta hacia la derecha.
9	$S \Rightarrow a c a c a c a g a c a c a$	P7	Avanza: Traza la arista horizontal superior (fondo del techo), dando ilusión 3D.

Caso D: Cruz Simétrica Radial

Paso	Cadena Derivada	Producción	Acción Geométrica (Semántica)
1	$S \Rightarrow \text{Cruz}$	P1	Llamada a la función de la figura.
2	$S \Rightarrow L R D L R D L R D L$	P5	Expansión del patrón radial en cruz.
3	$S \Rightarrow a R D L R D L R D L$	P7	Avanza: Dibuja la línea superior (brazo Norte).
4	$S \Rightarrow a t D L R D L R D L$	P10	Retorna: Vuelve al origen (nodo central) deslizando sobre la línea trazada.

5	$S \Rightarrow a t c L R D L R D L$	P8	Gira Der: Desde el origen, rota 90° (mira al Este).
6	$S \Rightarrow a t c a R D L R D L$	P7	Avanza: Dibuja el brazo Este.
7 a 11	$\dots \Rightarrow a t c a t c a R D L \dots$	P10, P8, P7	Se repite el retorno al centro, giro de 90° (mira al Sur) y trazo del brazo Sur.
Final	$S \Rightarrow a t c a t c a t c a$	P8, P7	Último retorno, giro (mira al Oeste) y traza el brazo final cerrando la simetría.

Caso E: Escalera (Zig-Zag)

Paso	Cadena Derivada	Producción	Acción Geométrica (Semántica)
1	$S \Rightarrow \text{Escalera}$	P1	Llamada a la función de la figura.
2	$S \Rightarrow L D L I L D L I L$	P6	Expansión del patrón alterno.
3	$S \Rightarrow a D L I L D L I L$	P7	Avanza: Traza la huella (base) del primer escalón.
4	$S \Rightarrow a c L I L D L I L$	P8	Gira Der: Se orienta hacia arriba.
5	$S \Rightarrow a c a I L D L I L$	P7	Avanza: Traza la contrahuella (altura) del escalón.
6	$S \Rightarrow a c a g L D L I L$	P9	Gira Izq: Se orienta nuevamente hacia el frente (restaura vector original).
7	$S \Rightarrow a c a g a D L I L$	P7	Avanza: Traza la huella del segundo escalón.
...	...	...	El patrón de giro derecha $\rightarrow$ izquierda mantiene el ascenso constante en el plano.
11	$S \Rightarrow a c a g a c a g a$	P7	Avanza: Traza el último peldaño, finalizando la iteración.

Higiene y Optimización de Gramáticas

Patologías de las Gramáticas Formales

En el diseño de compiladores, la calidad de una gramática formal no se limita a su capacidad matemática para generar cadenas. Si la estructura posee malformaciones, los analizadores sintácticos predictivos y descendentes colapsarán, generando bucles infinitos o interpretaciones de código inconsistentes. A continuación, se diagnostican y sanean tres casos prácticos fundamentales.

Gramática Ambigua

- **Concepto Teórico:** Una gramática es ambigua si existe al menos una cadena en su lenguaje que posee dos o más árboles de derivación independientes. En ingeniería de compiladores, la ambigüedad es una patología crítica: si el parser puede estructurar el código de dos formas distintas, el backend generará código de máquina impredecible, rompiendo la certeza del lenguaje.
- **Caso Práctico:** Expresiones aritméticas sin jerarquía de precedencia ni asociatividad explícita.
 - Alfabeto (Σ): { id, +, * }
 - Variables No Terminales (V_n): { E }
 - Reglas de Producción (P):
 - $E \rightarrow E + E \mid E * E \mid \text{id}$

Demostración de la Ambigüedad (Cadena de estrés: $id + id * id$): A continuación, se demuestra el fallo estructural modelando las dos interpretaciones jerárquicas totalmente divergentes que el compilador puede construir ante la misma entrada:

Árbol de Derivación A: Prioridad de la Multiplicación (Suma en la Raíz)

En esta estructura jerárquica, la suma se ubica en el nodo raíz del árbol, lo que obliga al compilador a evaluar primero los niveles inferiores. Semánticamente, esto equivale a interpretar la instrucción como $id + (id * id)$.

```
graph TD
    E1((E1)) --- E2((E2))
    E1 --- Op1("(" + ")")
    E1 --- E3((E3))

    E2 --- id1("[id]")

    E3 --- E4((E4))
    E3 --- Op2("(" * ")")
    E3 --- E5((E5))

    E4 --- id2("[id]")
    E5 --- id3("[id]")

    style E1 fill:#1e3a8a,stroke:#000,color:#fff
    style E3 fill:#3b82f6,stroke:#000,color:#fff
```

Árbol de Derivación B: Prioridad de la Suma (Multiplicación en la Raíz)

Debido a la patología de la gramática, el parser puede tomar una ruta de derivación sintáctica alterna, posicionando la multiplicación en el nodo raíz. Esto altera por completo la

jerarquía, forzando la evaluación anticipada de la suma, lo que equivale semánticamente a la expresión errónea $(id + id) * id$.

```
graph TD
    E1((E1)) --- E2((E2))
    E1 --- Op1("("*)
    E1 --- E3((E3))

    E2 --- E4((E4))
    E2 --- Op2("+")
    E2 --> E5((E5))

    E4 --- id1("[id]")
    E5 --- id2("[id]")

    E3 --- id3("[id]")

    style E1 fill:#1e3a8a,stroke:#000,color:#fff
    style E2 fill:#ef4444,stroke:#000,color:#fff
```

Conclusión del Análisis: La existencia demostrada de estos dos árboles estructurales para una misma palabra confirma la invalidez técnica de la gramática original para un compilador determinista, haciendo obligatoria su reescritura mediante la introducción de nuevos No Terminales que fijen la precedencia.

Recursividad por la Izquierda

- **Concepto Teórico:** Ocurre cuando una regla de producción permite que un No Terminal se invoque a sí mismo como el primer elemento de su expansión ($A \rightarrow A\alpha$). Esta malformación introduce un bucle infinito en los analizadores de descenso

recursivo, provocando un desbordamiento de la pila (Stack Overflow) al intentar expandir la variable antes de consumir cualquier terminal de entrada.

- **Caso Práctico:** Regla de acumulación sintáctica de términos.
- **Gramática Patológica:** $E \rightarrow E + T \mid T$

Algoritmo de Saneamiento: Para eliminar la recursividad inmediata de la forma $A \rightarrow A\alpha \mid \beta$:

1. El No Terminal A debe derivar en la base no recursiva β seguida de una nueva variable de control A'.
2. El nuevo No Terminal A' hereda el sufijo recursivo α seguido de sí mismo, cerrando el flujo con la transición vacía o Épsilon (ϵ).

- **Gramática Resultante Optimizada:**

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

Factorización por la Izquierda

- **Concepto Teórico:** Esta anomalía se presenta cuando un No Terminal tiene múltiples producciones que comparten el mismo prefijo común ($A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$). Al leer la entrada, un analizador sintáctico predictivo con un solo token de anticipación (Lookahead LL(1)) colisionará en un estado de no determinismo al no poder decidir cuál ruta expandir.
- **Caso Práctico:** El dilema del "Dangling Else" en condicionales anidados.
- **Gramática con No Determinismo:**
 $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$

El factor común que bloquea la decisión del parser es el prefijo "if E then S".

Algoritmo de Optimización:

1. Se extrae el factor común más largo ($\alpha = \text{if E then S}$).
2. Se reescribe la producción raíz para generar dicho factor acompañado de un nuevo No Terminal diferido (S'), manteniendo las producciones independientes (α).
3. El No Terminal secundario resolverá los sufijos restantes, incluyendo la opción vacía (ϵ).

- **Gramática Resultante Factorizada:**

$S \rightarrow \text{if E then } S S' \mid a$

$S' \rightarrow \text{else } S \mid \epsilon$

De la Expresión al Autómata (Caso Práctico:

Ajedrez)

Introducción y Contextualización Teórica

El estudio de los lenguajes formales y las gramáticas en este Tema 3 nos proporciona las herramientas matemáticas para entender cómo las computadoras procesan secuencias de símbolos con total certeza. Para materializar estos conceptos abstractos (alfabetos, palabras y reglas sintácticas), esta actividad propone la construcción de un analizador desde sus bases fundamentales mediante la dualidad técnica de la Expresión Regular y el Autómata Finito.

A nivel conceptual, estos dos términos representan las dos caras de la misma moneda en los lenguajes regulares (Tipo 3 de la Jerarquía de Chomsky):

- **La Expresión Regular** funciona como la definición descriptiva y algebraica del lenguaje; establece formalmente qué cadenas de texto pertenecen al sistema válido.
- **El Autómata Finito** es la contraparte operativa y algorítmica; representa la máquina abstracta ideal para leer caracteres en tiempo lineal y evaluar, paso a paso, si se cumplen las condiciones dictadas por la expresión.

El ajedrez se presenta como un escenario idóneo para aplicar esta teoría. Lejos de ser un juego caótico, el ajedrez se rige por un sistema estrictamente estructurado donde cada jugada representa una instrucción precisa. Para registrar estas partidas de forma universal, la comunidad informática y de ajedrez estandarizó la notación PGN (Portable Game Notation), la cual convierte las acciones del tablero en un lenguaje de texto con su propio alfabeto y morfología.

Sin embargo, para los objetivos de un analizador léxico puro, procesar la totalidad del PGN real implicaría una complejidad excesiva. El PGN completo incluye metadatos, comentarios anidados en paréntesis, variantes históricas y desambiguaciones que requieren memoria contextual, lo que desplazaría el problema hacia gramáticas más complejas (Tipo 2 o Tipo 1). Por lo tanto, para mantener el sistema dentro de las fronteras de un Lenguaje Regular puro, procesable por un escáner de memoria limitada y alta eficiencia, es metodológicamente necesario diseñar un subconjunto PGN simplificado. Este enfoque aísla la esencia morfológica de los movimientos y capturas básicos, permitiendo demostrar con rigor científico cómo se transita con éxito desde la abstracción de una expresión regular hasta la implementación física de un autómata en código.

Definición del Subconjunto PGN Simplificado

A partir del marco conceptual establecido, se delimitan las reglas formales del lenguaje regular específico que procesará nuestro sistema, reduciendo el espectro del PGN a sus componentes morfológicos atómicos.

Alfabeto Formal (Σ) y Clases de Caracteres

En la teoría de lenguajes, un alfabeto es un conjunto finito de símbolos. Sin embargo, modelar una transición matemática individual para cada pieza o columna resultaría en una explosión combinatoria inmanejable. Para simplificar nuestro modelo, el alfabeto original se ha optimizado agrupándolo en Clases de Caracteres, uniendo bajo una misma etiqueta los símbolos que comparten el mismo comportamiento morfológico:

- P (Piezas Mayores): [K, Q, R, B, N]
- C (Columnas): [a, b, c, d, e, f, g, h]
- F (Filas): [1, 2, 3, 4, 5, 6, 7, 8]
- X (Captura): [x]
- J (Jaque): [+]

Estructuras Léxicas Válidas: Basados en el alfabeto definido, las "palabras" aceptadas por nuestro lenguaje regular siguen cuatro estructuras base (con la adición opcional del estado terminal de jaque +):

1. Movimiento Simple de Peón: Columna + Fila (Ej: e4)
2. Movimiento Simple de Pieza: Pieza + Columna + Fila (Ej: Nf3)

3. Captura con Pieza: Pieza + x + Columna + Fila (Ej: Bxc6)

4. Captura con Peón: Columna Origen + x + Columna Destino + Fila Destino (Ej: exd5)

Proceso de Diseño de la Expresión Regular (Regex)

En la teoría de lenguajes, una Expresión Regular es una notación algebraica y declarativa que define exactamente qué cadenas pertenecen a un Lenguaje Regular (Tipo 3). No es una fórmula arbitraria, sino el resultado de un proceso de síntesis lógica.

Para diseñar nuestra Regex, no partimos de la fórmula final, sino que deconstruimos los casos de uso definidos en el subconjunto PGN mediante un enfoque "Bottom-Up" (de abajo hacia arriba):

Paso 1: El núcleo obligatorio (Destino): Todo movimiento de ajedrez, sin importar qué pieza se mueva o si hay captura, tiene un denominador común irrefutable: siempre debe indicar la casilla de destino. Por lo tanto, nuestro núcleo sintáctico es la concatenación de una columna y una fila:

Fragmento base: [a-h][1-8]

Paso 2: El prefijo de la pieza y la captura (Origen): A la izquierda de la casilla de destino, el lenguaje presenta variaciones morfológicas que debemos agrupar en una disyunción lógica (operador |):

- Si es una pieza mayor, inicia con su letra identificadora, seguida de una captura opcional: [KQRBN]x?
- Si es un peón que captura, inicia obligatoriamente con su columna de origen seguida de la x: [a-h]x

- Si es un peón avanzando, no hay prefijo (es vacío).
- Agrupamos estas posibilidades en un bloque opcional: $([KQRBN]x?[a-h]x)?$

Paso 3: El sufijo terminal (Jaque): Cualquier movimiento legal puede, de forma circunstancial, resultar en un jaque al rey oponente. Al ser opcional, añadimos el símbolo literal de suma al final con el cuantificador de cero o una ocurrencia:

Fragmento final: $+?$

Paso 4: Delimitación estricta de la cadena: Para que nuestro analizador léxico no acepte basura antes o después de la jugada (por ejemplo, rechazar "xxNf3yy"), anclamos la expresión al inicio (^) y al final (\$) de la lectura del token.

Resultado: La Expresión Regular Definitiva

Al concatenar los pasos lógicos anteriores, obtenemos la representación matemática exacta de nuestro subconjunto PGN:

$^([KQRBN]x?[a-h]x)?[a-h][1-8]+? \$$

Diseño y Profundización del Autómata Finito Determinístico (AFD)

Mientras que la Expresión Regular es una declaración estática de nuestro lenguaje, el Autómata Finito Determinístico (AFD) es el modelo dinámico y algorítmico que permite ejecutarla. En el diseño de compiladores, el AFD actúa como el motor del Analizador Léxico (Escáner).

Su propósito fundamental es leer el código fuente (en nuestro caso, el movimiento de ajedrez) carácter por carácter, transitando entre "estados" lógicos. Su mayor virtud

arquitectónica es su complejidad temporal lineal $O(n)$: al ser determinístico, por cada carácter que lee, la máquina sabe exactamente a qué único estado debe ir. No existe ambigüedad, no requiere una memoria de pila (stack) y jamás necesita retroceder (backtracking) para reevaluar la cadena.

Metodología de Diseño del Grafo

El diseño de nuestro AFD se construyó mapeando el flujo secuencial de la Expresión Regular hacia un grafo dirigido:

- 1. Estado Inicial (q_0):** Representa el sistema esperando el primer carácter. Desde aquí se abren dos caminos lógicos: leer una pieza mayor (P) que nos lleva a q_1 , o leer una columna minúscula (C) que nos lleva a q_2 .
- 2. Nodos de Tránsito (q_1, q_2, q_3, q_4):** Estos estados representan lecturas incompletas. Por ejemplo, en q_3 el escáner acaba de leer el símbolo de captura 'x', por lo que sabe que el siguiente carácter exigido por la gramática debe ser obligatoriamente una columna (C).
- 3. Estados de Aceptación (q_5, q_6):** Si la lectura de la cadena finaliza estando el autómata en uno de estos estados, el escáner léxico aprueba el token y se lo envía al analizador sintáctico/semántico. El estado q_5 representa un movimiento legal limpio, y q_6 representa un movimiento legal con jaque.
- 4. El Estado Sumidero (q_E):** Es el mecanismo de defensa del autómata. Si en cualquier estado se recibe un carácter inesperado (por ejemplo, leer dos 'x' seguidas), el sistema transita a q_E , un estado sin salida que garantiza el rechazo del código por Error Léxico.

Definición Formal (Quíntupla)

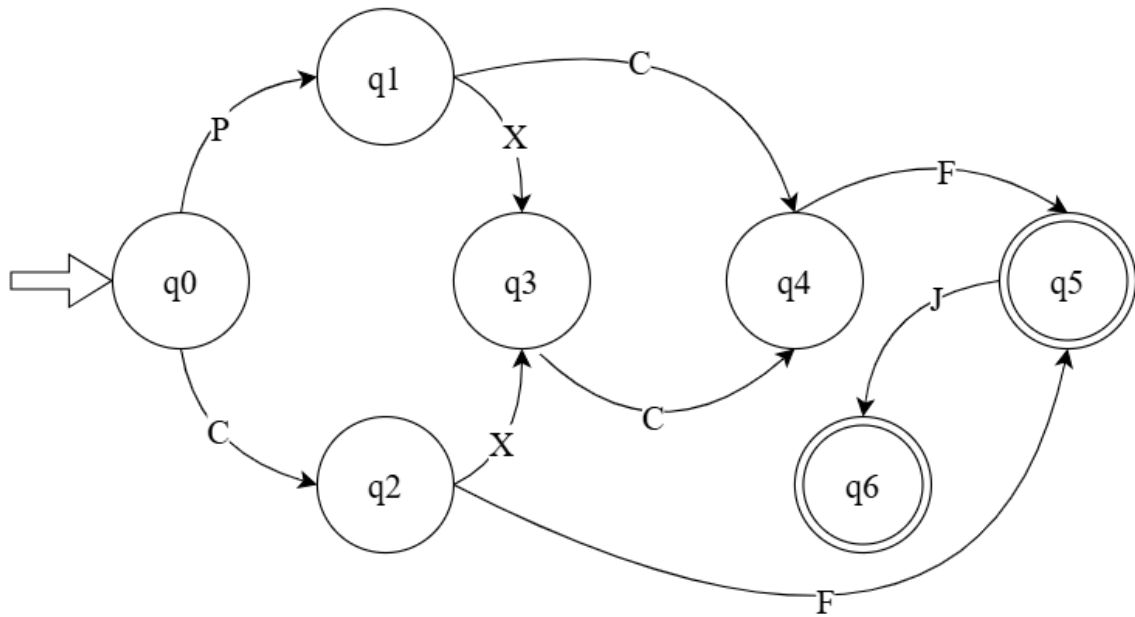
El AFD estructurado se define matemáticamente mediante $M = (Q, \Sigma, \delta, q_0, F)$:

- Q (Estados): $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_E\}$
- Σ (Alfabeto de Clases): $\{P, C, F, X, J\}$
- q_0 (Estado Inicial): q_0
- F (Estados de Aceptación): $\{q_5, q_6\}$

Función y Tabla de Transiciones (δ)

La siguiente matriz documenta el comportamiento determinístico del sistema. Si una celda indica q_E , significa que esa transición específica rompe las reglas sintácticas del PGN.

Estado Actual	Lee P (Pieza)	Lee C (Columna)	Lee F (Fila)	Lee X (Captura)	Lee J (Jaque)
$\rightarrow q_0$	q_1	q_2	q_E	q_E	q_E
q_1	q_E	q_4	q_E	q_3	q_E
q_2	q_E	q_E	q_5	q_3	q_E
q_3	q_E	q_4	q_E	q_E	q_E
q_4	q_E	q_E	q_5	q_E	q_E
* q_5	q_E	q_E	q_E	q_E	q_6
* q_6	q_E	q_E	q_E	q_E	q_E
q_E (Sumidero)	q_E	q_E	q_E	q_E	q_E



Nota Técnica sobre el alcance del AFD: Para preservar la legibilidad gráfica del autómata, el estado sumidero (q_E) ha sido omitido. Se asume implícitamente que cualquier transición no definida explícitamente conduce a q_E . La responsabilidad de este modelo es certificar la morfología estructural; las validaciones de coherencia espacial en el tablero corresponden a la fase de Análisis Semántico.

Ejemplo de Ejecución Paso a Paso:

A modo de demostración empírica, simularemos el recorrido del autómata al procesar la cadena Bxc6 (El Alfil captura en la casilla c6):

1. Inicio: El autómata arranca en el estado q_0 .
2. Lectura de 'B': El escáner clasifica el carácter 'B' dentro del grupo P (Piezas). Según la función de transición $\delta(q_0, P)$, el autómata avanza al estado q_1 .
3. Lectura de 'x': Se lee el símbolo literal 'x', clasificado en el grupo X (Captura). La transición $\delta(q_1, X)$ moviliza el sistema hacia el estado q_3 .

4. Lectura de 'c': El carácter 'c' pertenece al grupo C (Columnas). La transición $\delta(q3, C)$ dirige el flujo al estado $q4$.
5. Lectura de '6': El número '6' es clasificado en el grupo F (Filas). La transición $\delta(q4, F)$ lleva la máquina al estado $q5$.
6. Fin de Cadena (EOF): No hay más caracteres por leer. El sistema verifica su ubicación actual; como $q5$ pertenece al conjunto de estados de aceptación (F), el analizador léxico dictamina un Éxito y valida la palabra computacionalmente.

Caso contrario (Error Léxico): Si en el paso 3 el sistema hubiera leído un '9', este no pertenecería a ningún grupo válido del alfabeto esperado para ese estado, lo que invocaría una transición directa al estado sumidero (qE), rechazando la cadena irrevocablemente.

Conclusiones

La investigación y el desarrollo de los casos prácticos presentados en este informe reafirman que la teoría de lenguajes y gramáticas formales trasciende el ámbito puramente académico para convertirse en el pilar sobre el cual se sostiene la ingeniería de software moderna. A través de la Jerarquía de Chomsky, se comprende que cada nivel de abstracción sintáctica requiere un modelo computacional específico, determinando así la viabilidad y los límites de las herramientas informáticas que diseñamos.

El ejercicio de modelar el trazador gráfico demostró empíricamente cómo un conjunto estricto y finito de reglas de producción es capaz de derivar y controlar instrucciones algorítmicas complejas y cien por ciento predecibles. Por otro lado, el proceso de saneamiento gramatical dejó en evidencia que una arquitectura sintáctica deficiente o ambigua anula por completo la utilidad de un compilador determinista, subrayando que la optimización matemática de las reglas es un paso obligatorio antes de escribir la primera línea de código del analizador.

Finalmente, el diseño del analizador léxico para el ajedrez (PGN) comprobó la eficiencia de los Autómatas Finitos Determinísticos para validar cadenas en tiempo lineal computacional sin necesidad de recurrir a estructuras de memoria dinámica. En definitiva, dominar el diseño formal de gramáticas dota al ingeniero informático del criterio y la técnica necesarios para crear lenguajes escalables, seguros y exentos de contradicciones lógicas.

Referencias Bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.

[Compiladores, Principios, Tecnicas Y Herramientas | PDF | Informática y tecnología de la información | Science](#)

Brookshear, J. G. (1993). *Teoría de la computación: Lenguajes formales, autómatas y complejidad*. Addison-Wesley Iberoamericana.

[Brookshear. Teoria - De la - Computacion.lenguajes - Formales.automas.y.complejidad. .INCOMPLETO - Solo.capitulos.0.a.3. | PDF](#)

Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.

[\(PDF\) Introducción a La Teoría De Autómatas, Lenguajes Y Computación](#)

Sipser, M. (2007). *Introducción a la teoría de la computación* (2.^a ed.). Cengage Learning.

[Sipser - Introduction to the theory of computation - 2nd Ed.djvu](#)